

Game Registry: Class Diagram Description

Refer to the Structure (Class) Diagram for a visual representation of the game registry structure

GameRegistry

This will be the main class that lists all games we would have in the launcher; the static instance field holds a single copy of itself. The **game** field is the collection it manages. The methods below allow you to add games to the list (**register**), then allows to remove (**unregister**), look up games by their ID (**findById**), or check all games all at once (**listAll**). Everything else in the system that needs to know about goes through this class.

Game

This class describes everything the server knows about the game, a unique **id** to identify it, a **title** and **description** for display, and a **gameUrl** which is the actual web address if we post it on the web. The methods let us open the game in the browser (**launch**), pack the data into a (**GameDTO**) to send somewhere (**getMetadata**), or just get its URL (**getUrl**)

GameDTO

This is a read-only snapshot of a game's data used for transferring information; it will have no logic of its own, just public fields like **id**, **title**, and **gameUrl**. What produces this would be **Game.getMetadata()**.

LaunchConfig

This would hold the technical details needed when opening a game in the browser. It will belong to exactly one **Game** via composition. The **gameUrl** is where the game lives, and **openInNewTab** controls if the game will open in a new browser or on the same one and full screen controls for the full screen functionality. The **getUrl()** method will return the final url to navigate to, and the **validate()** checks that the config is complete and correct before trying to open anything.

Tag

This is a label class that is used to organize the games we would have in our system. For example, “multiplayer”, “puzzle”, or “simulator”. Each tag has an **id**, a **display label**, and a **colour** for the UI styling. The **getLabel()** method returns the tag's name, and **matches()** checks if the tag fits a search query. Tags will be shared across multiple games, so deleting a game doesn't delete the tags. This is why I used aggregation there.

Player

This will be someone who uses the launcher. Each player has an **Id**, **username**, **email**, and avatar **URL** for their profile picture. The **startSession()** method creates a new session, and **getHighScore()** takes a game Id and returns the player's best score for that specific game.

GameSession

A session is created when a player starts a game and represents that single playthrough. It tracks the players through their **playerId**, the game via **gameId**, the start time (**startedAt**), and the current score. The method **end()** closes the session when the player finishes. **submitScore()** sends the final score to the leaderboard, and **getDuration()** calculates how long the session lasted in seconds.

LeaderBoard

This will be a high score board for specific games; each game has exactly one. It stores the associated game via **gameId**, a collection of **LeaderBoardEntry** objects, a **lastUpdate** timestamp, and a **maxEntries** limit to keep it from growing indefinitely. The **addEntry()** method records a new score submission. **getTopR()** returns the top R cores for display, and **getRank()** takes a player id and returns their current position on the board.

LeaderBoardEntry

This is a record on a leaderboard; this represents one player's best score for a game. It stores their rank(1st, 2nd, 3rd), their **playerId**, and **playerName** for display, and their score. The **compareTo()** would be a helper method used to sort entries against each other when ranking is recalculated, and **toDisplay()** formats the entry as a readable string for the UI